AFRILANG Stdlib

std/c/statpercentile

Français. Bibliothèque AFRILANG ‘std/c/statpercentile‘ (niveau complex, catégorie complex). Elle expose 7 fonction(s) : percentile, percentiles, rankPct, decile, quantile, p90, p99.
‘import ”std/c/statpercentile”‘ · ‘use statpercentile‘
- ‘percentile(v list number, p number) → number‘ - ‘percentiles(v list number) → list number‘ - ‘rankPct(v list number, val number) → number‘ - ‘decile(v list number, d number) → number‘ - ‘quantile(v list number, q number) → number‘ - ‘p90(v list number) → number‘ - ‘p99(v list number) → number‘ **English.** AFRILANG library ‘std/c/statpercentile‘

(tier complex, category complex). Exports 7 function(s): percentile, percentiles, rankPct, decile, quantile, p90, p99.
‘import ”std/c/statpercentile”‘ · ‘use statpercentile‘
- ‘percentile(v list number, p number) → number‘ - ‘percentiles(v list number) → list number‘ - ‘rankPct(v list number, val number) → number‘ - ‘decile(v list number, d number) → number‘ - ‘quantile(v list number, q number) → number‘ - ‘p90(v list number) → number‘ - ‘p99(v list number) → number‘

Catégorie / Category	Modules complex (c/)
Niveau / Tier	complex
Fonctions / Functions	7

June 16, 2026

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/statpercentile' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : percentile, percentiles, rankPct, decile, quantile, p90, p99.

'import "std/c/statpercentile"' · 'use statpercentile'

```
- `percentile(v list number, p number) → number`
- `percentiles(v list number) → list number`
- `rankPct(v list number, val number) → number`
- `decile(v list number, d number) → number`
- `quantile(v list number, q number) → number`
- `p90(v list number) → number`
- `p99(v list number) → number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/statpercentile"
use statpercentile
```

```
Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés — graphes, NLP, réseaux complexes.
```

3. Référence API

```
•
 percentile(v list number, p number) → number•
 percentiles(v list number) → list•
 rankPct(v list number, val number) → number•
 decile(v list number, d number) → number•
 quantile(v list number, q number) → number•
 p90(v list number) → number•
 p99(v list number) → number
```

4. Exemples d'utilisation

```
import "std/c/statpercentile"
use statpercentile
```

```
create result = percentile(/* args */)
say result
```

```
create result = percentiles(/* args */)
say result
```

```
create result = rankPct(/* args */)
say result
```

```
create result = decile(/* args */)
say result
```

```
create result = quantile(/* args */)
say result
```

```
create result = p90(/* args */)
say result
```

5. Bonnes pratiques

```
•
 Préférez `import "std/c/statpercentile"` en tête de fichier.•
 Lancez `afrilang check` avant de livrer.•
 Combinez avec les modules stdlib de la même catégorie.•
 Voir /stdlib/ pour le source live et les démos playground.•
 Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe — source

module statpercentile

```
function percentile(v list number, p number) returns number
end
```

```
function percentiles(v list number) returns list number
end
```

```
function rankPct(v list number, val number) returns number
end
```

```
function decile(v list number, d number) returns number
end
```

```
function quantile(v list number, q number) returns number
end
```

```
function p90(v list number) returns number
end
```

```
function p99(v list number) returns number
end
end
```

English

1. Overview

AFRILANG library ‘std/c/statpercentile‘ (tier complex, category complex). Exports 7 function(s): percentile, percentiles, rankPct, decile, quantile, p90, p99.
‘import ”std/c/statpercentile”‘ · ‘use statpercentile‘

```
- `percentile(v list number, p number) → number`
- `percentiles(v list number) → list number`
- `rankPct(v list number, val number) → number`
- `decile(v list number, d number) → number`
- `quantile(v list number, q number) → number`
- `p90(v list number) → number`
- `p99(v list number) → number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/statpercentile"
use statpercentile
```

```
Tier: complex · Category: Complex modules (c/)
Advanced domains — graphs, NLP, complex networks.
```

3. API reference

```
•
 percentile(v list number, p number) → number•
 percentiles(v list number) → list•
 rankPct(v list number, val number) → number•
 decile(v list number, d number) → number•
 quantile(v list number, q number) → number•
 p90(v list number) → number•
 p99(v list number) → number
```

4. Usage examples

```
import "std/c/statpercentile"
use statpercentile
```

```
create result = percentile(/* args */)
say result
```

```
create result = percentiles(/* args */)
say result
```

```
create result = rankPct(/* args */)
say result
```

```
create result = decile(/* args */)
say result
```

```
create result = quantile(/* args */)
say result
```

```
create result = p90(/* args */)
say result
```

5. Best practices

```
•
 Prefer `import "std/c/statpercentile"` at the top of your file.•
 Run `afrilang check` before shipping.•
 Combine with related stdlib modules from the same category.•
 See /stdlib/ for the live source and playground demos.•
 Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix — source

module statpercentile

```
function percentile(v list number, p number) returns number
end
```

```
function percentiles(v list number) returns list number
end
```

```
function rankPct(v list number, val number) returns number
end
```

```
function decile(v list number, d number) returns number
end
```

```
function quantile(v list number, q number) returns number
end
```

```
function p90(v list number) returns number
end
```

```
function p99(v list number) returns number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/statpercentile"`
- Vérification : `afrilang check`
- Documentation : `https://afrilang.dev/stdlib/std/c/statpercentile/`

Source (extrait)

```
module statpercentile

function percentile(v list number, p number) returns number
end

function percentiles(v list number) returns list number
```

```
end

function rankPct(v list number, val number) returns number
end

function decile(v list number, d number) returns number
end

function quantile(v list number, q number) returns number
end

function p90(v list number) returns number
end

function p99(v list number) returns number
end
end
```